Executive Summary: 07-Treaps

1. Introduction

Many algorithmic applications require maintaining a dynamic ordered set of values while supporting fast updates and fast queries. Examples include ranking systems, live financial transaction monitoring, search indexes, event schedulers, and data streams where values are continuously inserted and removed. In these settings, the basic operations are usually:

- insert a key,
- delete a key,
- search for a key,
- answer order-related queries such as "what is the k-th smallest element?"

A simple unsorted array makes insertion easy but search expensive. A sorted array supports fast search via binary search, but insertion and deletion are expensive because many elements may need to be shifted. Linked lists avoid shifting costs, but searching remains linear. Ordinary binary search trees can be efficient when balanced, but in the worst case they may degenerate into a chain and require linear time.

Balanced binary search trees such as AVL trees and Red-Black trees address this by guaranteeing logarithmic height. However, these structures require relatively intricate rebalancing rules, with multiple structural cases and careful bookkeeping. While effective, they can be more difficult to implement and explain, especially for students seeing self-balancing search trees for the first time.

A **Treap** offers a different solution. A Treap combines:

- the **binary search tree property** on keys, and
- the **heap property** on random priorities.

Each node stores a key and a random priority. Keys determine in-order order, while priorities probabilistically keep the tree balanced. As a result, Treaps support expected $O(\log n)$ insertion, deletion, and search, while using a comparatively clean and elegant design.

The core problem addressed by Treaps is therefore:

> How can we maintain an ordered dynamic set efficiently, without relying on complicated deterministic balancing rules?

In this project, the target runtime for the main operations is expected $O(\log n)$. The main challenges are:

- preserving both the BST property and the heap property,
- implementing updates cleanly through structural operations,
- extending the data structure beyond basic search and update.

This final project focuses not only on standard Treaps, but also on an **augmented Treap** used in a custom programming challenge. In that challenge, students must extend the Treap to support additional analytics queries, including order-statistics and range aggregation. This turns the Treap from a basic balanced search tree into a richer data structure for real-time analysis.

2. Overall Intuitive Approach with Solution

The intuition behind a Treap is surprisingly simple: it is a binary search tree by *key*, but a heap by *priority*. Each node stores:

- a key, which determines where the node belongs in sorted order,
- a random priority, which influences the tree shape.

A valid Treap must satisfy two rules:

1. **BST rule:** all keys in the left subtree are smaller than the root key, and all keys in the right subtree are larger.
2. **Heap rule:** every node's priority is greater than the priorities of its children (assuming a max-heap convention).

The random priorities serve as a balancing mechanism. Rather than explicitly measuring subtree heights or storing color information, the Treap relies on randomness so that, in expectation, the tree remains balanced. This gives expected logarithmic height and therefore expected logarithmic runtime for the main operations.

A particularly elegant way to understand Treaps is through the two helper operations:

- **Split:** divide a Treap into two Treaps based on a key threshold,
- **Merge:** combine two Treaps when every key in the left Treap is less than or equal to every key in the right Treap.

These two operations give a very clean way to think about updates:

- insertion can be seen as either recursively descending like a BST or inserting a new root for a subtree and splitting around it,
- deletion can be seen as removing a node and merging its two children.

For the programming challenge, the Treap is augmented so it can answer more than simple membership queries.

3. Implementation

This section focuses on the implementation of the **original Treap** itself. The goal of the original Treap is to support the standard dynamic-set operations:

- `insert`
- `delete`
- `search`

in expected $O(\log n)$ time.

A Treap node stores:

- `key`: the value used for binary search tree ordering,

- `priority`: a randomly assigned number used for heap ordering,
- `left`, `right`: pointers to the left and right children.

The Treap must satisfy two invariants at all times:

1. **Binary Search Tree property:** all keys in the left subtree are smaller than the root key, and all keys in the right subtree are larger.
2. **Heap property on priorities:** each node's priority is greater than the priorities of its children, assuming a max-heap convention.

The priorities are chosen randomly when nodes are created. This randomization is what gives the Treap its expected logarithmic height. Intuitively, the random priorities prevent the tree from consistently degenerating into a long chain.

3.1 Node Structure

The basic Treap node only needs a small amount of information:

- the key,
- the priority,
- references to the left and right children.

A minimal node structure looks like this conceptually:

Listing 1: Basic Treap node structure

```
Node:
    key
    priority
    left
    right
```

When inserting a new key, a fresh random priority is assigned to the new node.

3.2 Search

Search in a Treap is identical to search in a standard binary search tree. The priorities do not affect the search path, because the keys alone determine whether the target lies in the left or right subtree.

If the target key matches the current root, the search succeeds. If the target key is smaller, the algorithm recurses left. If it is larger, the algorithm recurses right.

Listing 2: Pseudocode for search(root, key)

```
procedure SEARCH(root, key):
    if root is null:
        return false

    if root.key = key:
        return true

    if key < root.key:
        return SEARCH(root.left, key)
    else:
        return SEARCH(root.right, key)
```

Because the Treap has expected height $O(\log n)$, search runs in expected $O(\log n)$ time.

3.3 Split

One of the cleanest ways to implement Treaps is through the helper operation `split`. The operation `split(root, key)` divides a Treap into two smaller Treaps:

- one containing all keys less than or equal to `key`,
- one containing all keys greater than `key`.

The recursive structure is straightforward:

- if the current root key is less than or equal to the split key, then the root belongs to the left result, but its right subtree may contain keys on both sides of the split, so we recursively split the right subtree;
- otherwise, the root belongs to the right result, and we recursively split the left subtree.

Listing 3: Pseudocode for split(root, key)

```
procedure SPLIT(root, key):
    if root is null:
        return (null, null)

    if root.key <= key:
        (A, B) = SPLIT(root.right, key)
        root.right = A
        return (root, B)
    else:
        (A, B) = SPLIT(root.left, key)
        root.left = B
        return (A, root)
```

The split operation preserves both the BST property and the heap property in the resulting Treaps. This operation is central because it greatly simplifies insertion.

3.4 Merge

The companion operation to split is `merge`. The operation `merge(left, right)` assumes that every key in `left` is less than or equal to every key in `right`. Its purpose is to combine the two Treaps into one while preserving Treap validity.

The basic idea is:

- whichever root has higher priority becomes the root of the merged Treap,
- the merge continues recursively on the appropriate child.

Listing 4: Pseudocode for merge(left, right)

```
procedure MERGE(left, right):
    if left is null:
        return right
    if right is null:
        return left

    if left.priority > right.priority:
```

```
        left.right = MERGE(left.right, right)
        return left
    else:
        right.left = MERGE(left, right.left)
        return right
```

This works because the higher-priority root must remain above the lower-priority root in order to satisfy the heap condition, and the BST ordering is preserved by the assumption that all keys in the left Treap come before all keys in the right Treap.

3.5 Insertion

Insertion can be understood in two possible ways:

1. as binary-search-tree insertion followed by rotations,
2. or, more elegantly, using priorities together with split.

The split-based view is especially natural for Treaps. Suppose we want to insert a node with key `k` and a freshly generated random priority.

There are two cases:

- If the new node's priority is higher than the priority of the current subtree root, then the new node should become the root of that subtree. To make this happen, we split the old subtree by `k`. The left result becomes the new node's left child, and the right result becomes the new node's right child.
- Otherwise, the new node cannot become the root yet, so we descend left or right according to the BST property.

Listing 5: Pseudocode for insert(root, node)

```
procedure INSERT(root, node):
    if root is null:
        return node

    if node.priority > root.priority:
        (A, B) = SPLIT(root, node.key)
        node.left = A
        node.right = B
        return node

    if node.key < root.key:
        root.left = INSERT(root.left, node)
    else if node.key > root.key:
        root.right = INSERT(root.right, node)
    else:
        return root   // ignore duplicate in set version

    return root
```

This insertion algorithm preserves:

- BST ordering, because the split separates keys smaller than the inserted key from keys larger than it,
- heap ordering, because the new node becomes root only when its priority is larger.

Expected insertion time is $O(\log n)$ because the expected height of the Treap is logarithmic.

3.6 Deletion

Deletion is one of the most elegant parts of the Treap. Once the node to delete is found, we do not need a complicated balancing procedure. Instead, we simply remove the node and replace it by merging its left and right subtrees.

Thus, the algorithm is:

- search for the key using BST logic,
- when the node is found, merge its two children,
- return the merged subtree to the parent.

Listing 6: Pseudocode for delete(root, key)

```
procedure DELETE(root, key):
    if root is null:
        return null

    if root.key = key:
        return MERGE(root.left, root.right)

    if key < root.key:
        root.left = DELETE(root.left, key)
    else:
        root.right = DELETE(root.right, key)

    return root
```

This approach is much cleaner than the deletion logic in many deterministic balanced BSTs. The heap property is automatically restored by the merge operation, and the BST property is preserved because the merged children already respect key ordering.

3.7 Why Random Priorities Matter

A Treap's efficiency depends on the priorities being assigned randomly and independently. The random priorities effectively determine the shape of the tree. With high probability, the resulting structure has logarithmic height.

This randomized balancing is the main conceptual difference between Treaps and other balanced search trees. Instead of enforcing balance through strict structural rules, the Treap relies on probability. This makes the code simpler while still giving strong expected performance.

3.8 Complexity

Let n be the number of keys currently stored in the Treap. Because the expected height of the Treap is $O(\log n)$, the following operations each take expected $O(\log n)$ time:

- `search`
- `insert`
- `delete`
- `split`

- `merge`

The space usage is $O(n)$, since each node stores a constant amount of information.

4. Summary of Programming Challenge

The programming challenge developed for this project is framed as a real-time fraud analytics problem. Students are told to imagine they are processing a stream of credit card transaction values and must support efficient real-time queries on the active set of transactions.

The challenge assumes that the standard Treap operations `INSERT`, `DELETE`, and `SEARCH` are already implemented. Students are then required to extend the Treap to support:

- `KTH(k)`: return the k-th smallest element,
- `COUNT(x)`: return the number of elements less than or equal to x,
- `RANGE(l, r)`: return the number of elements in $[l, r]$,
- `SUM(l, r)`: return the sum of all elements in $[l, r]$.

The learning objectives of this assignment are:

1. understand how Treaps combine randomization with BST ordering,
2. practice augmenting a data structure with subtree metadata,
3. learn how structural operations such as split, merge, insertion, and deletion must maintain that metadata,
4. see how a theoretical data structure can be adapted to support practical analytics tasks.

The key twist in the assignment is that students are not simply implementing a standard Treap. Instead, they must transform it into an **augmented Treap** capable of order-statistics and range aggregation. This goes beyond the simple version of a Treap and forces students to think more deeply about how local metadata supports global queries.

At the same time, the challenge is still at an appropriate level for an upper-level undergraduate course (since we can solve it by ourselves). The extension is meaningful, but it remains manageable because it builds directly on the Treap framework students have already studied.

5. Key Ideas for Solving Programming Challenge

The most important idea for solving the programming challenge is that the new queries can all be supported through subtree summaries.

For `KTH`, `COUNT`, and `RANGE`, the key metadata is **subtree size**. This allows the algorithm to skip entire subtrees instead of exploring every node.

For `SUM`, the extra metadata is **subtree sum**. This lets the algorithm aggregate all keys in a subtree immediately when that entire subtree falls inside the query condition.

A second important idea is that metadata maintenance is the real challenge. The query functions themselves are relatively short. What makes the problem interesting is that students must correctly update:

- `size`
- `subtreeSum`

after every structural modification. If these values are wrong at even one node, the advanced queries may return incorrect answers.

A third important idea is reducing range queries to prefix-style queries:

$$\text{RANGE}(l, r) = \text{COUNTLEQ}(r) - \text{COUNTLEQ}(l - 1)$$

$$\text{SUM}(l, r) = \text{SUMLEQ}(r) - \text{SUMLEQ}(l - 1)$$

This is elegant because it avoids writing a more complicated direct range traversal. It also highlights a common algorithmic pattern: transform a difficult query into two simpler ones.

These changes are interesting because they show how flexible Treaps are. The students do not invent a brand-new data structure, but they do meaningfully extend an existing one. The result is a data structure that behaves like a compact analytics engine for dynamic ordered data.

The model solution runs in expected $O(\log n)$ time per operation, which is efficient enough for large input sizes such as 2×10^5 operations.

6. Conclusion

This project studied Treaps as a randomized balanced binary search tree that combines the binary search tree property on keys with the heap property on priorities. Treaps provide an elegant alternative to more rigid balanced trees because they achieve expected logarithmic performance with comparatively simple structural logic.

The project also emphasized that Treaps are not just useful for basic insertion, deletion, and search. Through augmentation, they can support richer queries such as order statistics and range aggregation while preserving the same expected asymptotic efficiency. By storing subtree size and subtree sum at each node, an augmented Treap can answer `KTH`, `COUNT`, `RANGE`, and `SUM` queries efficiently.

The custom programming challenge was designed to help students see exactly this point. Instead of merely reproducing the basic data structure, they must adapt it to satisfy new requirements. This makes the assignment more interesting, more realistic, and more educationally valuable.

Overall, Treaps are a strong example of how randomness, recursive structure, and careful metadata design can work together to produce a data structure that is both theoretically elegant and practically useful.